BLOODHOUND — Beginner's Battle Plan

Goal: An AI agent that retrospectively hunts an attacker already inside a network, finds them by writing and running real queries, reconstructs the attack timeline, and makes a room of senior developers go quiet.

Your edge: Everyone else fakes it. You build the real thing and can prove it live. That is the entire strategy. Read the next section twice.

THE ONE RULE THAT BEATS SENIOR DEVS

Senior developers win arguments by asking *"okay, but what if I change the input?"* They have seen a thousand demos that fall apart the second they go off-script. Their instinct is to find the seam where you faked it.

So you remove the seam.

You build it for real, in a *constrained* way that is reliable. Then when a judge says "run it again on different data," you smile and run it. That moment — surviving the adversarial question — is worth more than any animation. A beginner who built something genuinely real beats a senior who built something brittle and impressive-looking.

Translation of every "fake this" tip into "build this for real":

The lazy version (loses)	The real version you'll build (wins)
Hardcode MITRE mappings	Let the model classify behavior → technique ID for real (it's reliable for clear behaviors)
Pre-script the queries	Template-constrained queries the model *selects* based on live evidence
Mock the Elastic cluster	Run a real local Elasticsearch in Docker (more reliable AND more honest)
Fake the streaming text	Real token streaming over SSE from the model
One golden dataset only	A second *clean* dataset you can run live to prove it's not rigged

---

TOOLS & AI AGENTS — what to actually use

You're a beginner, so let AI agents do the heavy lifting on code while you stay in control of the *architecture* and the *story*. Here's the stack and who does what.

AI coding agents (these write most of your code)

- **Claude Code** (terminal/desktop agent) — your main builder. Give it one task at a time: "build the synthetic data generator following this schema." It writes, runs, and fixes the code. Best for multi-file work and debugging.
- **Cursor** or **Windsurf** (AI code editors) — alternative if you prefer an IDE with inline AI. Good for learning because you see every change.
- Pick **one** main agent and stick with it. Switching tools mid-build wastes days.

The "brain" of your product (the reasoning LLM inside BLOODHOUND)

- **Gemini API** (Gemini 2.x Flash or Pro) — your roadmap already chose this. Flash is cheap and fast (good for the loop), Pro is smarter (good for the final report). Start with Flash.
- You will need a free Google AI Studio API key. Get this on Day 1 so it's not a blocker later.

Orchestration & backend

- **LangGraph** — controls the investigation loop (hypothesis → query → evidence → branch). Don't fear it; it's just a state machine. You'll learn the 5 functions you need.
- **FastAPI** — your backend server + the streaming endpoint.
- **Pydantic** — forces the model's output into a fixed shape so it can't return garbage. This is a beginner's best friend.

Search layer

- **Elasticsearch** (single node, via Docker) — stores the logs, runs the EQL.
- **Python Elasticsearch client** — how your backend talks to it.

Frontend

- **React + Vite** (scaffold it with the AI agent, don't hand-build).
- **TailwindCSS** — styling without writing CSS files.
- **React Flow** — the lateral-movement graph (the tweetable part).
- **Framer Motion** — animations (edges drawing, findings pinning).

Everything-else

- **Docker + Docker Compose** — runs Elasticsearch with one command.
- **Git/GitHub** — version control + a clean public repo is itself a credibility weapon.

REALISTIC TIMELINE

The original doc assumes a 7-day sprint by an experienced team. **You're a beginner, so the honest version is 10–12 days.** Below is a 10-day plan with the work, plus what to *learn* each day (just-in-time, never ahead). If you truly only have 7 days, follow the "7-DAY COMPRESSION" note at the end.

Phase	Days	What you produce
0. Setup & learning runway	Day 0 (½ day)	Working environment, all accounts, "hello world" of each tool
1. Synthetic attack dataset	Days 1–2	500k realistic log events with the attack hidden in noise
2. Search layer	Day 3	Local Elasticsearch + EQL template library + executor
3. The reasoning loop	Days 4–5	The agent that hunts by itself (the core)
4. Timeline + MITRE	Day 6	Chronological narrative + real technique mapping
5. Blast radius + IR report	Day 7	"14,000 records, 2.3GB" + the report
6. The interface	Days 8–9	The detective board — streaming, graph, timeline
7. Polish + rehearsal	Day 10	First 10 seconds perfected, 20+ run-throughs

---

DAY 0 — SETUP (do not skip, this prevents 3 future disasters)

Goal: every tool says "hello world" before you build anything real.

1. Install: Python 3.11+, Node.js (LTS), Docker Desktop, Git, your AI editor (Cursor or Claude Code).
2. Get a **Gemini API key** from Google AI Studio. Run one test call from a Python script. Confirm it works.
3. Run Elasticsearch in Docker once: confirm `curl http://localhost:9200` returns JSON. (Your AI agent can give you the exact `docker-compose.yml`.)
4. Create a **GitHub repo** named `bloodhound`. Commit an empty project. You'll commit every day.
5. Make a `notes.md` file. Every number you'll claim in the demo (2.3GB, 14k records, 4 minutes) gets its math written here so you can defend it instantly.

Learn today (90 min max): what an API key is, what Docker does (runs software in a box so it "just works"), what a terminal command is. Ask your AI agent to explain any error in plain English.

Beginner trap: Don't try to understand everything. Understand *enough to run the next command*. Depth comes later.

DAYS 1–2 — THE DATASET (your credibility foundation)

This is where teams lose security judges. Over-invest here. If the data looks fake, nothing else matters.

What you're building: a Python script that generates ~500,000 log events across 6 days, using **real Elastic ECS field names**, with the attack woven *into* realistic noise.

Required real field names (use these exactly): `@timestamp`, `event.action`, `event.category`, `source.ip`, `source.geo.country_name`, `user.name`, `host.name`, `network.bytes`, `process.name`

The noise (this is what makes it believable):

- 200 normal users with realistic daily rhythm — busy 9–6, quiet at night, *some* legit after-hours work.
- Routine failed logins (people fat-finger passwords).
- Scheduled backups that move large amounts of data (so a big transfer isn't automatically "the attacker").
- Occasional vulnerability scans.

The attack thread (hidden inside the noise):

1. `mark.chen` — credentials phished.
2. 2:14 AM auth (off-hours, but not alone — others work late too).
3. Same user authenticates from Seattle, then London, 20 minutes later (impossible travel).
4. Lateral movement: hop to a database host.
5. 2.3GB outbound at 2:47 AM (the exfil).

The rule: the attack must NOT be the only weird thing in the data. The agent finding it *among* believable noise is the whole proof.

Tools: Python, the `Faker` library (random realistic names/IPs/times), `json`/`csv`. Let your AI agent write the generator — you specify the schema and the attack story above, it writes the code.

Learn today: what JSON looks like, what a timestamp/timezone is, what an IP address is. That's it.

Definition of done: you can open the data file and the attack is genuinely hard to spot by eye. If you can spot it in 5 seconds, add more noise.

DAY 3 — SEARCH LAYER (make the queries real and unbreakable)

Goal: load the data into Elasticsearch and build the query system that *can't* produce broken syntax.

1. **Load** the 500k events into local Elasticsearch (AI agent writes the ingestion script).
2. **Build the EQL template library — 12–15 validated templates.** Each is a correct EQL query with blanks to fill. Examples:
   - Off-hours authentication for a user
   - Impossible travel (same user, two countries, short time gap)
   - Lateral movement (auth chain across hosts)
   - Large outbound transfer (network.bytes over threshold)
   - Unusual process spawn
3. **Validate every template** by running it against the live index. If it returns results correctly, it's locked in.
4. **Build the executor:** a Python function that takes `(template_id, params)` → runs it → returns the real JSON result.

Why this beats senior devs: raw LLM-generated EQL fails ~30% of the time. By making the model *choose and fill* a validated template instead of writing raw syntax, every query is guaranteed valid. Your defensible one-liner:

> "We constrain the model to a validated query DSL — it selects and parameterizes, it never writes raw EQL. The intelligence is in *which* query to run next based on the last result, not in generating syntax."

Learn today: what a database query is (a question you ask your data), what EQL roughly does (find security events matching a pattern). You do NOT need to master EQL — the templates handle it.

Definition of done: you can call `execute(template_id, params)` from Python and get real results back, every time, no errors.

DAYS 4–5 — THE REASONING LOOP (the heart of the product)

Goal: the agent hunts *by itself* — and genuinely branches based on what it finds.

The loop (built with LangGraph):

```
seed alert
  → form hypothesis
  → select template + fill params
  → execute query (real Elasticsearch)
  → read the REAL JSON result
  → interpret + decide next hypothesis
  → branch or conclude
```

The non-negotiable design rule: after each query, the model receives the **actual JSON result** and must output:

- its interpretation of what it found,
- the next hypothesis,
- the next `template_id` + `params`.

Query #3 must genuinely depend on what query #2 returned. If the data were different, it would hunt differently. Build it so this is *literally true* — then you can say it out loud and survive the "change the input" question.

Use Pydantic to force the model's output into a fixed shape:

```
{ interpretation: str, next_hypothesis: str, template_id: str, params: dict }
```

If the model returns anything else, you reject and retry. This is your reliability seatbelt.

Test in the terminal first — no UI yet. It must reach the exfil event in ⩽4 queries across 10+ runs. Don't move on until this is boringly reliable. Reliability here is the whole game.

Which model: Gemini Flash for the loop (fast, cheap, runs many times during testing).

Learn these 5 days: what a "state machine" is (a thing that moves between steps and remembers where it is), what "structured output" means (forcing JSON), what a prompt is. LangGraph is just: define the steps, define how they connect. Your AI agent scaffolds it; you understand the flow.

Beginner trap: Don't let the loop run forever. Cap it at 6 iterations max. A runaway loop in front of judges is death.

DAY 6 — TIMELINE + MITRE (the emotional payload + the credibility)

Timeline reconstruction: take the query results and assemble a chronological narrative with **exact timestamps, IPs, hosts, affected resources**. Not a summary paragraph — a real timeline:

```
02:14 AM  mark.chen authenticated  (host: WS-4471, Seattle)
02:14 AM  mark.chen authenticated  (London)  ← impossible travel
02:31 AM  lateral movement → DB-PROD-02
02:47 AM  2.3GB outbound  ← exfiltration
```

Real MITRE mapping (do NOT hardcode): feed the model the relevant ATT&CK technique list and let it map observed behaviors → technique IDs. The behaviors here are unambiguous (impossible travel, lateral movement, exfil), so the model maps them correctly *and consistently* across runs. That's why doing it for real is safe — and why you survive "run it again."

Learn today: what MITRE ATT&CK is (a public dictionary of attacker techniques with IDs like T1078). 20 minutes of reading is enough.

Definition of done: timeline + technique IDs come out correct on 5 consecutive runs.

DAY 7 — BLAST RADIUS + IR REPORT

Blast radius: from the exfil event, estimate what data was accessed using DB query logs in the same time window. Make it *math you can defend:*

```
records × avg_record_size = total bytes ≈ 2.3GB → ~14,000 customer records
```

Write this formula in `notes.md`. When a judge asks "where's 2.3GB from?", you answer in one sentence. That's the difference between credible and hand-wavy.

IR report: generate a short enterprise-style incident report (executive summary, patient zero = mark.chen, timeline, blast radius, containment recommendation). Use Gemini Pro here for quality. Output as clean Markdown (you can convert to PDF later if you want).

Learn today: nothing new. Consolidation day.

DAYS 8–9 — THE INTERFACE (where "damn" lives)

Not a dashboard. A detective's investigation board. Dark, terminal aesthetic.

Three live regions:

1. **Left — streaming reasoning.** Real token streaming over **SSE** (Server-Sent Events) from FastAPI. The model's thinking scrolls live. *This is your entertainment and it kills dead air —*

there's never an 80-second spinner because text is always moving.

2. **Center — lateral movement graph** (React Flow). Edges draw between hosts as each hop is discovered. This is the part that gets screenshotted and tweeted.
3. **Right — timeline** building chronologically, each finding pinned with a confidence indicator. Framer Motion for the pin/draw animations.

Credibility detail: show the **real Elastic JSON response for ~2 seconds** before each interpretation. Raw API data on screen = "this is talking to real Elastic, not a mock."

Build approach: scaffold the whole React app with your AI agent. You direct the layout and behavior; it writes the components. Connect frontend to the FastAPI SSE stream.

Learn these 2 days: what a frontend/backend split is, what "streaming" means (data arriving piece by piece, not all at once), roughly what a React component is. You will not master React — the agent builds it, you steer.

Beginner trap: Don't redesign 5 times. Pick the dark-terminal look, lock it, move on. Polish > novelty.

DAY 10 — POLISH + REHEARSAL (first impression is most of the score)

The first 10 seconds decide the room. Build this opening sequence:

```
silent scrolling log feed
  → "Days Since Last Alert: 6" counter on screen
  → BLOODHOUND activates
  → first anomaly flares RED
```

No narration needed. The visual tells the story.

Then rehearse the full run 20+ times. Each time:

- Kill every console error.
- Eliminate layout shift (things jumping around).
- Pre-warm the Elasticsearch connection and the model *before* you present (cold starts are slow and ugly).

Build the secret weapon: a **second clean dataset** — same 200 users, same noise, NO attack. Be ready to run the hunt on it live. It concludes "no significant findings." Nothing destroys "it's all hardcoded" suspicion like watching your agent honestly come up empty on clean data. Most teams can't do this. You can, because you built it real.

THE DEMO SCRIPT (memorize, do not improvise)

- **0:00** "This is your SOC dashboard from the last 7 days. Clean week. Everyone went home happy." Pause. "They shouldn't have."
- **0:15** "BLOODHOUND runs a retrospective hunt every 24 hours. Here's what it found at 3 AM." Agent starts.
- **0:30** Finding 1: off-hours auth. "Not unusual by itself. Watch what comes next."
- **0:55** Finding 2: impossible travel, edge draws on graph. "You can't be in both places."
- **1:20** Finding 3: lateral movement → DB → 2.3GB out. "14,000 customer records." **Say nothing. Let it land.**
- **1:40** Close: "mark.chen's credentials were stolen 6 days ago. Every alert was satisfied. Your SIEM didn't find this in 6 days. BLOODHOUND found it in 4 minutes." **Hard stop. 3 seconds of silence. Look at the judges, not the screen.**

The one architecture answer to rehearse (a dev judge WILL ask "how do you keep the EQL from breaking?"):

> "We constrain the model to a validated query DSL — it chooses and parameterizes, it never writes raw EQL. The reasoning is in which query to run next based on the last result, not in generating syntax."

SMALL DETAILS THAT MAKE DEVS RESPECT YOU

- Clean browser console during the demo (open it briefly on purpose — signals confidence).
- Real `@timestamp` values with timezones, never round numbers.
- A visible `curl http://localhost:9200` at some point — proves Elastic is real, not mocked.
- Monospace font for queries/JSON, sans-serif for narration.
- No spinner ever runs longer than the streaming text covers it.
- A GitHub repo open in a tab with real daily commit history.
- A clean README with an architecture diagram and a one-command Docker setup. Expert devs trust code they could clone and run.

WHAT TO CUT — RUTHLESSLY

- Dynamic mapping of arbitrary novel attacks.
- Real-time network packet capture.
- Authentication, multi-tenancy, scalability.

- The "detection rules to prevent recurrence" feature.
- Anything that takes more than 2 seconds to render mid-demo.

Every one of these dilutes the single story: **it hunted the attacker by itself, and you can verify it live.**

7-DAY COMPRESSION (if you truly can't get 10 days)

- Days 1–2 → compress to Day 1–2 but use a smaller dataset (100k events, still noisy).
- Buy back time by leaning harder on your AI agent and skipping the IR report PDF (keep it as on-screen Markdown).
- Day 6 MITRE + Day 7 blast radius → combine into one day.
- Protect the loop reliability days and the polish day at all costs. A reliable boring demo beats a flashy broken one.

ORDER OF OPERATIONS (if you forget everything else)

1. Data first (everything depends on it, judges scrutinize it hardest).
2. Make queries reliable before making them smart.
3. Make the loop reliable before making it pretty.
4. Build the second clean dataset — it's your shield.
5. Polish the first 10 seconds last, but polish them more than anything else.

Build it real. Prove it live. That's how a beginner makes a room of senior developers go quiet.